PRÁCTICA EXTRA: DICCIONARIOS Y CONJUNTOS

Estructuras de Datos: dict y set — Ejercicios Complementarios

Objetivo de la Práctica: Ejercitar las operaciones de diccionarios (dict) y conjuntos (set) y decidir cuál conviene según el problema.

Estilo y Buenas Prácticas (PEP 8): *snake_case* para variables y funciones, *SCREAMING_SNAKE_CASE* para constantes.

Metodología Polya: Comprender el problema, planificar la estructura de datos, codificar y verificar el resultado por pantalla.

SECUENCIA DE PROBLEMAS PRÁCTICOS

PROBLEMA 1: Diccionario básico

Consigna: Crear `equipo = {"nombre": "PC-Lab01", "ram_gb": 8, "so": "Debian"}`.
a) Mostrar `len` y el valor de `"so"`.
b) Pedir `"disco"` con `.get()` (¿qué devuelve?).
c) Cambiar `ram_gb` a 16, insertar `disco_gb` y agregar con `.update()` `sede` y `estado`.
d) Verificar con `in` que existe `"sede"`.
e) Borrar `"estado"` con `del`.
f) Recorrer con `.items()` e imprimir `clave: valor`.

Fase Polya / Guía de Resolución:

1. ¿Cuáles son los datos de entrada, las constantes y la salida esperada?
2. ¿Qué estructura (for, dict, set, lista) conviene usar y por qué?
3. ¿Qué resultado esperás obtener con los datos del enunciado?
4. Espacio para boceto de código:

PROBLEMA 2: Contar frecuencias

Consigna: `sistemas = ["Linux","Windows","Linux","Linux","Windows","macOS"]`. Con un `for` y un diccionario vacío, contar cuántas veces aparece cada sistema. *Esperado: Linux 3, Windows 2, macOS 1.*

Fase Polya / Guía de Resolución:

1. ¿Cuáles son los datos de entrada, las constantes y la salida esperada?
2. ¿Qué estructura (for, dict, set, lista) conviene usar y por qué?
3. ¿Qué resultado esperás obtener con los datos del enunciado?
4. Espacio para boceto de código:

PROBLEMA 3: Nómina por legajo

Consigna: `nomina` es un diccionario cuyas claves son legajos y cuyos valores son diccionarios `{"nombre", "nota"}` (Brisa 8.5, Abel 7.0, Celeste 9.5). Constante `NOTA_PROMOCION = 8.0`.
a) Mostrar la nota de Abel por acceso directo.
b) Recorrer con `.items()`, acumular la suma y avisar quiénes promocionan.
c) Mostrar promedio y cantidad de promocionados. *Esperado: 7.0, promedio 8.33, 2 promocionados.*

Fase Polya / Guía de Resolución:

1. ¿Cuáles son los datos de entrada, las constantes y la salida esperada?
2. ¿Qué estructura (for, dict, set, lista) conviene usar y por qué?
3. ¿Qué resultado esperás obtener con los datos del enunciado?
4. Espacio para boceto de código:

PROBLEMA 4: Conjuntos

Consigna: `asistencias = ["Brisa","Abel","Celeste","Brisa","Diego","Abel"]` e `inscriptos = {"Abel","Diego","Erika","Fabi"}`. Obtener `presentes = set(asistencias)` y mostrar (con `sorted`): cantidad de presentes, intersección `&`, unión `|`, diferencia `inscriptos - presentes` y diferencia simétrica `^`.

Fase Polya / Guía de Resolución:

1. ¿Cuáles son los datos de entrada, las constantes y la salida esperada?
2. ¿Qué estructura (for, dict, set, lista) conviene usar y por qué?
3. ¿Qué resultado esperás obtener con los datos del enunciado?
4. Espacio para boceto de código:

PROBLEMA 5: ¿Set o dict?

Consigna:
a) `sin_repetidos(lista)` devuelve los elementos únicos ordenados.
b) `buscar_precio(catalogo, producto)` devuelve el precio o `-1` si no existe (usar `.get(clave, -1)`). Probar con una lista de IPs repetidas y un catálogo de 3 productos. Justificar por escrito por qué cada caso usa `set` o `dict`.
Soluciones en `problema_N_*.py`.

Fase Polya / Guía de Resolución:

1. ¿Cuáles son los datos de entrada, las constantes y la salida esperada?
2. ¿Qué estructura (for, dict, set, lista) conviene usar y por qué?
3. ¿Qué resultado esperás obtener con los datos del enunciado?
4. Espacio para boceto de código: